

# Frontend · Mundo Pokémon — Fase 2 · Maquetación CSS

## Ejercicio de cumplimiento obligatorio

Esta práctica es de **cumplimiento obligatorio**, ya que se utilizará como referencia de evaluación continua y servirá de base para las fases posteriores de JavaScript y React.

## Propósito de esta fase

En esta segunda fase vas a trabajar la **maquetación CSS** de la interfaz de Mundo Pokémon a partir del HTML que construiste en la fase 1.

Aquí ya no toca diseñar la estructura del documento desde cero, sino:

- dar forma visual a la interfaz;
- transformar el HTML en una composición clara y coherente;
- aplicar layout, espaciado, tipografía, color e imágenes;
- y acercarte de forma razonable a la referencia visual sin depender de copiarla al píxel.

## Contexto de la práctica transversal

Este ejercicio forma parte de una práctica transversal del bloque de frontend. La misma interfaz evolucionará más adelante en:

- **Fase 3 · JavaScript**
- **Fase 4 · React**

## Qué se va a trabajar aquí

En esta fase se evalúa sobre todo:

- la capacidad de convertir una estructura HTML en una interfaz visual consistente;
- el uso razonable de variables, tipografía, color, espaciado y sombras;
- la composición del layout con las herramientas CSS trabajadas en el bloque;
- el tratamiento de imágenes dentro de tarjetas;

- la adaptación responsive de la interfaz;
- y la organización del CSS para que siga siendo mantenible.

### Recurso opcional de refuerzo · CSS Grid Garden

Si todavía no controlas bien CSS Grid o necesitas visualizar mejor cómo funcionan columnas, filas y colocación dentro de una rejilla, puedes reforzar conocimientos jugando a **CSS Grid Garden**:

<https://cssgridgarden.com/#es>

Úsalo como apoyo visual y de comprensión, pero recuerda no dedicar demasiado tiempo ahí: **lo más importante de esta práctica es aplicar conocimientos reales de maquetación en una interfaz completa.**

Si quieres, puedes retomar el juego en otro momento para seguir afianzando conceptos.

---

## 1. Competencias que vas a trabajar

Al terminar esta fase deberías ser capaz de:

- enlazar y organizar correctamente una hoja de estilos en un proyecto real;
- transformar una estructura HTML previa en una interfaz visual clara;
- usar variables CSS para dar consistencia a color, espaciado y radios;
- escoger con criterio las herramientas de layout según el problema;
- trabajar tarjetas repetibles con imágenes, chips y contenido textual;
- integrar `object-fit`, `aspect-ratio`, sombras, bordes y estados visuales cuando corresponda;
- adaptar la interfaz a distintos anchos de pantalla de forma razonable;
- y construir una base CSS suficientemente sólida para añadir comportamiento en la fase de JavaScript.

---

## 2. Organización del trabajo

### 2.1. Carpeta de trabajo

Trabaja sobre la misma carpeta del proyecto:

```
frontend/  
└─ mundo-pokemon/  
    └─ index.html  
    └─ styles.css
```

### Importante

En esta fase no vas a crear una maqueta aislada distinta. Vas a seguir trabajando sobre el mismo proyecto iniciado en la fase 1.

## 2.2. Recomendación de rama

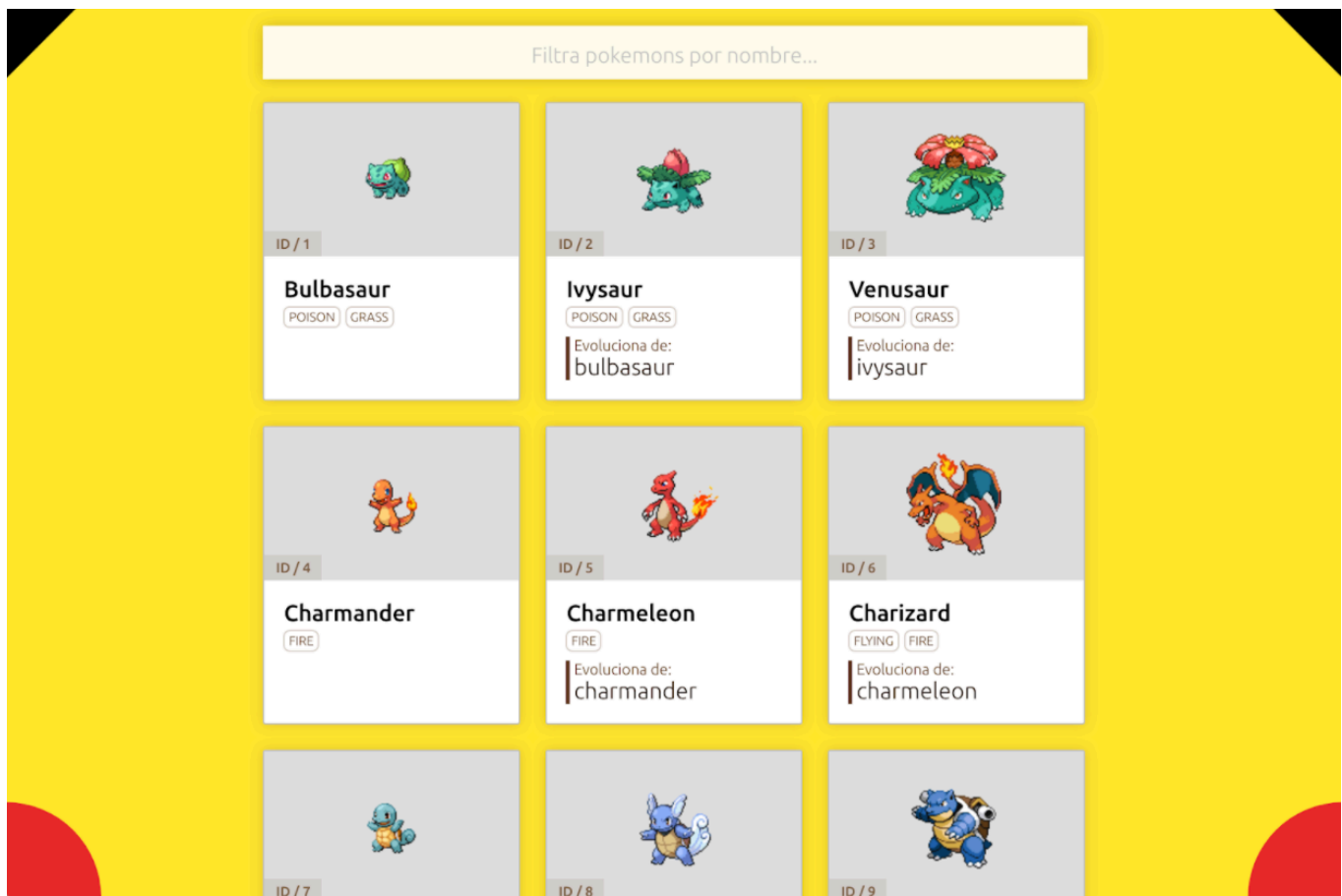
Si vas a trabajar con ramas, usa una rama específica para esta fase.

Ejemplo recomendado:

```
feature/frontend-mundo-pokemon-phase-02-css
```

---

## 3. Material de referencia



### 3.1. Referencia visual

La imagen muestra una interfaz tipo Pokédex con estos elementos principales:

- un fondo dominante llamativo;
- una caja superior de búsqueda;
- una rejilla de tarjetas;
- una composición centrada;
- tarjetas con imagen, identificador, nombre, tipos y, en algunos casos, evolución;
- y algunos elementos decorativos geométricos en las esquinas.

### 3.2. Qué debes interpretar de la referencia

No se espera que la reproduzcas al píxel, pero sí que construyas una interfaz que:

- recuerde claramente a la composición de referencia;
- mantenga una jerarquía visual parecida;
- tenga una secuencia o rejilla de tarjetas coherente;
- y aplique criterios razonables de spacing, layout y consistencia visual.

No copies “formas” sin entender su función. Antes de escribir CSS, identifica:

- qué parte corresponde al contenedor general;
- qué parte hace de zona de búsqueda;
- qué parte hace de secuencia o rejilla de tarjetas;
- y qué elementos de la interfaz son estructurales y cuáles son decorativos.

### 3.3. Sobre la tipografía de referencia

Para esta práctica puedes usar **Ubuntu** como tipografía principal (Usa Google Fonts).

Trabaja con distintos pesos para reforzar la jerarquía visual de la interfaz. Por ejemplo:

- un peso más marcado para el nombre del Pokémon;
- un peso medio o regular para textos secundarios;
- y un peso más discreto para elementos auxiliares como identificadores o evolución.

---

## 4. Reglas generales de esta fase

En esta fase:

- trabaja con una hoja de estilos externa;
- usa variables CSS siempre que ayuden a mantener consistencia;
- no intentes resolver toda la interfaz solo con `position`;
- utiliza las herramientas vistas en teoría con criterio, no por obligación mecánica;
- usa `gap` si el espacio forma parte del layout;
- no conviertas el CSS en una lista desordenada de reglas sueltas;
- y recuerda que una interfaz limpia vale más que una copia visual desordenada.

#### Error frecuente

En una práctica visual como esta es fácil obsesionarse con “que se parezca mucho”.

Aquí el orden correcto es:

1. construir primero una base visual coherente;
2. resolver layout y espaciado con claridad;
3. adaptar después la interfaz a distintos anchos;

4. y solo después acercarte más a los detalles visuales.

## 5. Tarea de la fase

Debes construir el archivo `styles.css` necesario para maquetar la interfaz de Mundo Pokémon a partir del HTML ya existente.

El resultado debe encajar de forma razonable con la imagen de referencia y resolver, como mínimo, estas partes de la interfaz:

- una composición general clara;
- una caja superior de búsqueda visualmente priorizada;
- una secuencia ordenada de tarjetas;
- una estructura visual interna consistente para cada tarjeta;
- una presentación diferenciada para los tipos;
- una integración razonable de las imágenes;
- y una adaptación responsive básica pero funcional.

Debes trabajar especialmente:

- jerarquía tipográfica;
- fondos de zonas internas;
- espaciados;
- bordes o sombras;
- integración de la imagen;
- consistencia entre tarjetas;
- y comportamiento de la interfaz cuando el ancho disponible cambia.

### Prioridad de la práctica

Lo importante no es la decoración.

Lo importante es que la maquetación esté resuelta de manera adecuada siguiendo la teoría que has trabajado a lo largo del bloque de CSS: variables, layout, imágenes, tipografía, jerarquía, responsive y acabado visual.

## 6. Responsive

La interfaz debe responder de forma razonable cuando el ancho disponible cambie. No se espera una batería enorme de breakpoints ni una solución exageradamente compleja, pero sí que la composición:

- no dependa solo de un ancho de escritorio;
- reorganice bien la secuencia de tarjetas;
- mantenga legibilidad en tamaños más reducidos;
- y conserve una jerarquía visual clara.

### Alcance de esta fase

No se espera todavía comportamiento real de filtrado ni generación dinámica de tarjetas. Esta fase sigue siendo puramente de maquetación CSS.

### Qué se espera aquí

No se trata solo de “hacer que quepa”.

Se trata de que la interfaz siga funcionando visualmente bien cuando cambie el espacio disponible.

---

## 7. Orientaciones de trabajo

### Cómo abordar la práctica

Una estrategia razonable puede ser esta:

1. crea primero una base global de `body`, contenedor principal y tipografía;
2. maqueta después la zona de búsqueda;
3. resuelve la zona general de tarjetas;
4. trabaja una sola tarjeta hasta que funcione bien;
5. comprueba después que todas las demás heredan bien el mismo patrón;
6. y por último revisa cómo debe adaptarse la interfaz cuando cambia el ancho disponible.

## 🔥 En lo primero que conviene centrarse: la tarjeta

Igual que en la fase HTML la tarjeta era el patrón estructural más importante, aquí vuelve a ser el patrón visual más importante.

Antes de entrar en detalles decorativos, revisa:

- ¿la tarjeta tiene una estructura interna clara?;
- ¿la imagen se integra bien?;
- ¿el nombre destaca lo suficiente?;
- ¿los tipos se entienden como chips?;
- ¿el bloque de evolución está bien separado del resto?;
- ¿las tarjetas mantienen consistencia aunque no todas tengan exactamente el mismo contenido?

## 🔥 Buen criterio

Si una tarjeta está bien resuelta, el resto de esa secuencia de tarjetas suele avanzar mucho mejor.

Si una tarjeta está mal resuelta, el problema se multiplica nueve veces.

## ✍ Organización del CSS

Intenta agrupar los estilos por bloques lógicos de la interfaz. Por ejemplo:

- base global;
- contenedor principal;
- zona de búsqueda;
- secuencia de tarjetas;
- tarjeta individual;
- detalles internos;
- y adaptaciones responsive.

No es obligatorio que uses exactamente ese orden, pero sí conviene que el archivo resulte legible y fácil de seguir.



## 8. Criterios de calidad

El ejercicio se considerará bien resuelto si:

- el CSS está organizado y resulta legible;
- la composición general recuerda claramente a la referencia;
- la caja de búsqueda queda bien situada y visualmente priorizada;
- la secuencia de tarjetas está bien distribuida;
- cada tarjeta tiene una jerarquía visual clara;
- las imágenes se integran con coherencia;
- los tipos se perciben como etiquetas diferenciadas;
- la interfaz mantiene consistencia entre tarjetas;
- se aprecia uso razonable de variables CSS;
- la adaptación responsive está resuelta de forma funcional y coherente;
- la organización del CSS refleja bloques lógicos de la interfaz y no una acumulación de reglas sin relación;
- y el resultado queda preparado para pasar a la fase de JavaScript sin necesidad de rehacer la maquetación.

---

## 9. Cierre

Al terminar esta fase deberías haber convertido tu estructura HTML inicial en una interfaz visual clara, coherente, adaptable y suficientemente sólida para evolucionar después con JavaScript y React.

### Idea final

Una buena maquetación no consiste en “hacer que se parezca”. Consiste en tomar buenas decisiones de layout, jerarquía visual, adaptación y consistencia para que la interfaz funcione como sistema.